architecture_doc.pdf

Date: September 5, 2026 Prepared for: Final-Year B.Tech CSE (AIML) Student Tech Stack Focus: FastAPI, PostgreSQL, JWT Auth, SQLAlchemy, Vector DB, RAG Pipeline Your project idea is highly relevant and market-aligned for 2026. The Python backend ecosystem has matured around FastAPI as the de-facto standard for AI/ML APIs, and RAG (Retrieval-Augmented Generation) has become the baseline architecture for production LLM applications. This report breaks down exactly what you need to learn, in what or

for production LLM applications. This report breaks down exactly what you need to learn, in what order, and how to execute this project to build both strong backend engineering skills and modern AI/ML competencies. FastAPI Dominance: FastAPI has become the preferred framework for AI/ML backends due to its async support, automatic OpenAPI documentation, Pydantic validation, and dependency injection system. Job postings for Python backend roles in 2026 explicitly require FastAPI experience alongs

n system. Job postings for Python backend roles in 2026 explicitly require FastAPI experience alongside async programming and microservices architecture. Async-First Architecture: Modern Python backends are async by default. SQLAlchemy 2.0's async ORM with asyncpg driver for PostgreSQL is now the production standard, replacing synchronous patterns. Key practices include: One AsyncSession per request/task (never shared across coroutines) expire_on_commit=False to prevent runtime errors in async c

st/task (never shared across coroutines) expire_on_commit=False to prevent runtime errors in async context Dependency injection for session management via FastAPI's Depends() Clean Architecture Patterns: Production codebases use repository-service-router separation with explicit dependency injection. This matches your planned architecture exactly. Professional Learning & Execution Report: AI Knowledge Base Backend with RAG Executive Summary [1] [2] [3] 1. Market Landscape & Technology Trends (20

edge Base Backend with RAG Executive Summary [1] [2] [3] 1. Market Landscape & Technology Trends (2025-2026) 1.1 Python Backend Development [2] [4] [5] [1] [6] [7] [8] [9] [6] [10] [11] [6] [4] [11] [12] [7] [11] RAG is Production Standard: Retrieval-Augmented Generation has moved from research to production baseline. Every enterprise LLM application in 2026 uses some form of RAG to ground responses in domain-speci■c data. Vector Database Landscape: The vector DB market has consolidated around a

es in domain-speci■c data. Vector Database Landscape: The vector DB market has consolidated around a few key players: Database Best For Scale Setup Cost pgvector Teams already on Postgres, <50M vectors Millions Low (extension) Free Qdrant Speed + metadata filtering, self-hosted 10M+ Medium Free tier Chroma Prototyping, MVPs <5M Very Low Free Weaviate Hybrid keyword + semantic search Millions Medium Free tier Pinecone Fully managed, hands-o■ ops Any scale Low Paid Recommendation for your project:

m Free tier Pinecone Fully managed, hands-o■ ops Any scale Low Paid Recommendation for your project: Start with pgvector since you're already using PostgreSQL. It handles up to ~50M vectors comfortably, inherits your backup/DR setup, and requires zero additional infrastructure. Switch to Qdrant or Weaviate only if you exceed 10M vectors or need advanced hybrid search. RAG Frameworks: Two dominant orchestration frameworks exist: Framework Strength Best For LlamaIndex Retrieval-first, document-cen

orchestration frameworks exist: Framework Strength Best For LlamaIndex Retrieval-first, document-centric Knowledge bases, Q&A over PDFs/docs LangChain Agent-centric, tool-heavy Multi-step workflows, agents Recommendation: Use LlamaIndex for your knowledge base project. It reaches a working RAG system in ~5 lines, has better defaults for chunking/hybrid search, and is optimized for document Q&A. LangChain with

LangGraph is better suited for agentic work■ows you plan to add later. Document Process

ngChain with LangGraph is better suited for agentic work■ows you plan to add later. Document Processing: IBM's Docling (MIT-licensed, open source) has emerged as the production standard for document ingestion in 2026. It handles PDF, DOCX, TXT, Markdown with hierarchical chunking that preserves document structure (headings, tables, sections) as metadata—critical for high-quality retrieval. Embedding Models: The MTEB leaderboard tracks state-of-the-art embedding models. For 2026: BGE-M3 (BAAI): B

ng Models: The MTEB leaderboard tracks state-of-the-art embedding models. For 2026: BGE-M3 (BAAI): Best multilingual, supports 100+ languages E5-large-v2 (Microso): Strong English performance SentenceTransformers (HuggingFace): Easy local deployment Use the same embedding model for indexing and querying—vectors from di■erent models are not comparable. 1.2 AI/LLM/RAG Ecosystem [13] [14] [15] [16] [14] [17] [18] [19] [20] [21] [22] [13] [15] [17] [18] [19] [20] [21] [16] [23] [24] [25] [26] [23] [

14] [17] [18] [19] [20] [21] [22] [13] [15] [17] [18] [19] [20] [21] [16] [23] [24] [25] [26] [23] [24] [25] [27] [28] [29] [30] [26] [13] [26] AI Agents & Tool Calling: For future agentic features, the landscape includes: LangGraph (LangChain): Stateful, graph-based agent work■ows CrewAI: Role-based multi-agent collaboration AutoGen (Microso): Conversable agents with tool use Must-Have Before Starting: 1. FastAPI Fundamentals Path operations, request/response models Pydantic models for validati

rting: 1. FastAPI Fundamentals Path operations, request/response models Pydantic models for validation Dependency injection (Depends()) Async/await patterns Exception handlers, middleware 2. SQLAlchemy 2.0 Async ORM create_async_engine with postgresql+asyncpg:// async_sessionmaker with expire_on_commit=False ORM models with Mapped[] type annotations select() queries with await session.execute() or await session.scalars() Relationships with selectinload()/joinedload() (no lazy loading in async) A

await session.scalars() Relationships with selectinload()/joinedload() (no lazy loading in async) Alembic for migrations 3. JWT Authentication with OAuth2 OAuth2PasswordBearer security scheme Password hashing with bcrypt or passlib JWT token creation/veri■cation with python-jose or PyJWT Access tokens + refresh tokens Dependency for get_current_user 4. Repository-Service Architecture Repository layer: CRUD operations, session-aware Service layer: Business logic, orchestration Router layer: HTTP

ayer: CRUD operations, session-aware Service layer: Business logic, orchestration Router layer: HTTP endpoints, request/response models Dependency injection chain: Router → Service → Repository → Session Learning Resources: [27] [28] 2. Prerequisites & Skills Required 2.1 Core Backend Skills (FastAPI + PostgreSQL) [5] [4] [8] [11] [12] [31] [6] [7] [10] [32] [33] [34] [35] [11] [7] FastAPI o■cial docs (fastapi.tiangolo.com) SQLAlchemy 2.0 async tutorial "FastAPI + JWT/OAuth2 from Scratch" guides

docs (fastapi.tiangolo.com) SQLAlchemy 2.0 async tutorial "FastAPI + JWT/OAuth2 from Scratch" guides Conceptual Understanding Required: 1. What is an LLM? Large Language Model: Transformer-based neural network trained on massive text corpora Generates text autoregressively (token-by-token) Limitations: Hallucinations, knowledge cuto■, no access to your private data 2. What is RAG? Retrieval-Augmented Generation: Augments LLM prompts with retrieved context from your data Four-step pipeline: 1. In

ted Generation: Augments LLM prompts with retrieved context from your data Four-step pipeline: 1. Ingest: Load documents → chunk → embed → store vectors 2. Retrieve: Embed query → similarity search → shortlist chunks 3. Rerank (optional): Cross-encoder reranking for precision 4. Augment + Generate: Build grounded prompt → LLM → cited answer Why RAG? Reduces hallucinations, grounds responses in your data, enables citations 3. Embeddings & Vector Search Embedding: Dense vector representation of te

r data, enables citations 3. Embeddings & Vector Search Embedding: Dense vector representation of text (e.g., 768-dim for BGE-M3) Semantic similarity: Cosine similarity between vectors Vector index: HNSW (approximate nearest neighbor) for fast retrieval 4. Document Chunking Strategies Fixed-size chunking: 512 tokens with 10-20% overlap (baseline) Semantic chunking: Split at topic boundaries using embedding distance thresholds Hierarchical chunking: Preserve document structure (headings, sections

embedding distance thresholds Hierarchical chunking: Preserve document structure (headings, sections) as metadata Best practice: 512 tokens, 50-token overlap, attach source metadata (■lename, page, section) Learning Resources: "RAG Explained" guides LlamaIndex documentation (docs.llamaindex.ai) LangChain RAG tutorials [8] [10] [35] 2.2 AI/ML Concepts (LLM, RAG, Embeddings) [36] [37] [16] [23] [13] [26] [30] [38] [39] [23] [36] [26] [39] [23] [36] [38] [39] [29] [30] [24] [23] [30] [36] [40] [16]

[26] [30] [38] [39] [23] [36] [26] [39] [23] [36] [38] [39] [29] [30] [24] [23] [30] [36] [40] [16] Backend Stack: fastapi>=0.115.0 uvicorn[standard] sqlalchemy>=2.0.0 asyncpg alembic pydantic>=2.0.0 python-jose[cryptography] # or PyJWT passlib[bcrypt] python-multipart # for ■le uploads AI/RAG Stack: llama-index-core llama-index-llms-openai # or llama-index-llms-ollama for local llama-index-embeddings-huggingface # or openai llama-index-vector-stores-pgvector llama-index-readers-■le # PDF, DOCX

dings-huggingface # or openai llama-index-vector-stores-pgvector llama-index-readers-■le # PDF, DOCX, TXT, Markdown docling # IBM's document parser sentence-transformers # local embeddings Vector Database: pgvector # PostgreSQL extension # OR qdrant-client # if you switch later Utilities: python-dotenv # environment variables httpx # async HTTP client pytest # testing Goal: Solidify FastAPI + SQLAlchemy async patterns before adding AI complexity. Tasks: 1. Build a minimal FastAPI app with: User

lchemy async patterns before adding AI complexity. Tasks: 1. Build a minimal FastAPI app with: User registration/login (JWT auth) Protected routes (/users/me) 2.3 Libraries & Tools to Master 3. Learning Path & Execution Plan Phase 0: Foundation (Week 1-2) CRUD for a simple resource (e.g., notes, tasks) Async SQLAlchemy with AsyncSession dependency Alembic migrations 2. Implement clean architecture: routers/, services/, repositories/, models/, schemas/ Dependency injection chain Error handling, l

: routers/, services/, repositories/, models/, schemas/ Dependency injection chain Error handling, logging Deliverable: A working authentication + CRUD backend (no AI yet). Goal: Understand and implement a basic RAG pipeline in isolation. Tasks: 1. Learn LlamaIndex basics: Document loaders (PDF, TXT, Markdown) Text splitters (chunking with overlap) Embedding models (HuggingFace or OpenAI) Vector stores (start with in-memory or Chroma for prototyping) Query engines (retrieve + generate) 2. Build

tores (start with in-memory or Chroma for prototyping) Query engines (retrieve + generate) 2. Build a standalone RAG script: Load a sample PDF Chunk with SentenceSplitter (512 tokens, 50 overlap) Embed with HuggingFaceEmbedding (BGE-M3 or E5-large-v2) Store in Chroma/FAISS Query and print responses with source citations 3. Experiment with chunking strategies: Fixed-size vs. semantic chunking Impact of overlap on retrieval quality Metadata attachment (■lename, page number) Deliverable: A Jupyter

t of overlap on retrieval quality Metadata attachment (■lename, page number) Deliverable: A Jupyter notebook or script demonstrating end-to-end RAG over a sample document. Phase 1: RAG Fundamentals (Week 3-4) Goal: Integrate RAG into your FastAPI backend with pgvector. Tasks: 1. Set up pgvector: Install pgvector extension on PostgreSQL Create vector column in SQLAlchemy model HNSW index for fast similarity search 2. Design database schema: users table (existing) knowledge_bases table (user_id, n

imilarity search 2. Design database schema: users table (existing) knowledge_bases table (user_id, name, created_at) documents table (kb_id, ■lename, ■le_path, uploaded_at) chunks table (document_id, text, embedding, metadata_json, chunk_index) conversations table (user_id, kb_id, title, created_at) messages table (conversation_id, role, content, cited_chunk_ids, created_at) 3. Implement ingestion pipeline: File upload endpoint (POST /knowledge-bases/{kb_id}/documents) Background task (Celery or

pipeline: File upload endpoint (POST /knowledge-bases/{kb_id}/documents) Background task (Celery or FastAPI BackgroundTasks) to: Parse document with Docling or LlamaIndex readers Chunk with hierarchical splitter Generate embeddings Insert chunks into pgvector 4. Implement chat endpoint: POST /knowledge-bases/{kb_id}/chat Retrieve top-k chunks via pgvector similarity search Build grounded prompt with context Call LLM (OpenAI or local Ollama) Return response with cited chunk IDs Save conversation

ith context Call LLM (OpenAI or local Ollama) Return response with cited chunk IDs Save conversation to database Deliverable: Working API with document upload, ingestion, and chat with citations. Goal: Add production-grade features and scalability. Tasks: Phase 2: Integration (Week 5-7) Phase 3: Production Hardening (Week 8-10) 1. Streaming responses: Use StreamingResponse with Server-Sent Events (SSE) for token-by- token LLM output. 2. Hybrid search: Combine pgvector similarity with PostgreSQL

(SSE) for token-by- token LLM output. 2. Hybrid search: Combine pgvector similarity with PostgreSQL full-text search for better recall. 3. Reranking: Add a cross-encoder reranker (e.g., BGE-reranker) to improve precision of top-k results. 4. Rate limiting: Implement per-user rate limits on chat endpoints. 5. Observability: Add structured logging, request tracing, and metrics (latency, token usage). 6. Testing: Write pytest tests for: Authentication ■ows Document ingestion RAG retrieval accuracy

. 6. Testing: Write pytest tests for: Authentication ■ows Document ingestion RAG retrieval accuracy API contract validation Deliverable: Production-ready backend with streaming, hybrid search, and tests. Goal: Add agentic work■ows and advanced AI capabilities. Tasks: 1. Memory & context: Implement conversation memory (summarize long conversations, retrieve relevant history). 2. Tool calling: Integrate LangGraph or CrewAI for multi-step work■ows (e.g., "search my docs, then summarize, then email

rate LangGraph or CrewAI for multi-step work■ows (e.g., "search my docs, then summarize, then email the result"). 3. Multi-tenancy: Add row-level security in PostgreSQL for tenant isolation. 4. Evaluation: Implement RAGAS or custom metrics to measure retrieval quality (Recall@K, MRR, NDCG). 5. Deployment: Dockerize the app, write Kubernetes manifests, deploy to cloud (AWS/GCP/Azure). Deliverable: Agentic RAG system with evaluation metrics and deployment scripts. 1. Sharing AsyncSession across co

Agentic RAG system with evaluation metrics and deployment scripts. 1. Sharing AsyncSession across coroutines: Never store AsyncSession on a long-lived service object. Create a new session per request/task. 2. Lazy loading in async: Set lazy="raise" on relationships to catch accidental lazy loads at development time. [17] [15] [37] Phase 4: Advanced Features (Week 11-12+) [28] [27] [15] [13] 4. Common Pitfalls & Best Practices Backend Pitfalls [6] [7] [9] [6] 3. Missing expire_on_commit=False: Wi

mon Pitfalls & Best Practices Backend Pitfalls [6] [7] [9] [6] 3. Missing expire_on_commit=False: Without this, accessing ORM attributes aer commit raises errors in async context. 4. Undersized connection pool: Async can saturate a small pool quickly. Start with pool_size equal to expected concurrent DB operations, not worker count. 1. Wrong embedding model for indexing vs. querying: Vectors from di■erent models are not comparable. Use the same model for both. 2. Naive chunking: Fixed-size chunk

di■erent models are not comparable. Use the same model for both. 2. Naive chunking: Fixed-size chunking without overlap or metadata loses context. Use 512 tokens with 50-token overlap and attach source metadata.

3. No reranking: Top-k similarity search can return irrelevant chunks. Add a cross-encoder reranker for production quality. 4. Hallucinated citations: Force the LLM to cite only retrieved chunks. Use prompt engineering or constrained decoding to prevent fake citations. 5. Ignoring docume

chunks. Use prompt engineering or constrained decoding to prevent fake citations. 5. Ignoring document structure: Flat chunking loses headings, tables, sections. Use Docling or LlamaIndex's hierarchical chunker to preserve structure. 1. Password hashing: Use bcrypt with appropriate work factor (12+). 2. JWT expiration: Short-lived access tokens (15-30 min) + refresh tokens (7-30 days). 3. Input validation: Validate ■le types, sizes, and content before processing. 4. Rate limiting: Protect chat e

lidation: Validate ■le types, sizes, and content before processing. 4. Rate limiting: Protect chat endpoints from abuse with per-user rate limits. FastAPI O■cial Docs:https://fastapi.tiangolo.com SQLAlchemy 2.0 Async Tutorial:https://universopython.com/en/blog/python-sqlalchemy-async FastAPI + JWT Auth Guide:https://mjmjmj.name/python/fastapi-jwt-oauth2-authentication/ Async SQLAlchemy Patterns:https://www.querystack.tech/post/sqlalchemy-20-async-orm-pattern s-pitfalls-and-production-ready-sessi

tps://www.querystack.tech/post/sqlalchemy-20-async-orm-pattern s-pitfalls-and-production-ready-session-management-5c4068 LlamaIndex Docs:https://docs.llamaindex.ai RAG Explained Guide:https://khimananda.com/blog/rag-explained-retrieval-augmented-generatio n Building Production RAG with LlamaIndex:https://blog.redlineso.net/posts/building-productio n-ready-rag-pipeline-llamaindex/ [10] [11] [6] [6] RAG Pitfalls [26] [23] [24] [36] [37] [37] [30] [29] Security Best Practices [35] [35] 5. Recommend

RAG Pitfalls [26] [23] [24] [36] [37] [37] [30] [29] Security Best Practices [35] [35] 5. Recommended Learning Resources FastAPI + SQLAlchemy [7] [35] [6] RAG & LlamaIndex [36] [24] LangChain RAG Tutorial:https://pratikdhanave.com/blog/posts/lc-05-retrieval-and-rag.html pgvector GitHub:https://github.com/pgvector/pgvector Vector DB Comparison 2026:https://www.guideflow.com/blog/vector-search-soware Best Vector Databases for RAG:https://modelrefs.com/tools/best-vector-databases/ Docling GitHub:h

re Best Vector Databases for RAG:https://modelrefs.com/tools/best-vector-databases/ Docling GitHub:https://github.com/DS4SD/docling Web Scraping for RAG:https://www.context.dev/blog/how-to-scrape-websites-into-llm-ready-mark down-for-rag-pipelines LangGraph Docs:https://langchain-ai.github.io/langgraph/ CrewAI Docs:https://docs.crewai.com LangGraph vs CrewAI vs AutoGen:https://dev.to/priyeshdave6/langgraph-vs-crewai-vs-autogen-1 07-real-data-engineering-tasks-at-scale-4h3e 1. Week 1-2: Build aut

langgraph-vs-crewai-vs-autogen-1 07-real-data-engineering-tasks-at-scale-4h3e 1. Week 1-2: Build auth + CRUD backend (no AI). 2. Week 3-4: Learn RAG with LlamaIndex in a notebook. 3. Week 5-7: Integrate RAG into FastAPI with pgvector. 4. Week 8+: Add streaming, hybrid search, agents. Master async SQLAlchemy patterns before adding RAG complexity. Understand chunking, embeddings, and retrieval before optimizing. Build a working MVP ■rst, then harden for production. Write README with architecture d

e optimizing. Build a working MVP ■rst, then harden for production. Write README with architecture diagrams. Comment your code explaining RAG pipeline decisions. Create a blog post or LinkedIn article about your learnings (great for internships). [16] Vector Databases [41] [14] Document Processing [29] [30] AI Agents [27] 6. Final Recommendations Start Simple, Iterate Fast Focus on Fundamentals Document Everything You already know Python and PostgreSQL—this is a natural extension. Your AI/ML cou

cument Everything You already know Python and PostgreSQL—this is a natural extension. Your AI/ML coursework gives you conceptual grounding; now apply it practically. This project is directly relevant to 2026 job postings for Python backend + AI roles. ai-knowledge-base/ ■■■ app/ ■ ■■■ __init__.py ■ ■■■ main.py # FastAPI app, routers, middleware ■ ■■■ con■g.py # Settings, env vars ■ ■■■ database.py # Async engine, session factory ■ ■■■ models/ ■ ■ ■■■ __init__.py ■ ■ ■■■ user.py ■ ■ ■■■ knowledge

Async engine, session factory ■ ■■■ models/ ■ ■ ■■■ __init__.py ■ ■ ■■■ user.py ■ ■ ■■■ knowledge_base.py ■ ■ ■■■ document.py ■ ■ ■■■ chunk.py ■ ■ ■■■ conversation.py ■ ■ ■■■ message.py ■ ■■■ schemas/ ■ ■ ■■■ __init__.py ■ ■ ■■■ user.py ■ ■ ■■■ knowledge_base.py ■ ■ ■■■ document.py ■ ■ ■■■ chat.py ■ ■■■ repositories/ ■ ■ ■■■ __init__.py ■ ■ ■■■ user.py ■ ■ ■■■ knowledge_base.py ■ ■ ■■■ ... ■ ■■■ services/ ■ ■ ■■■ __init__.py ■ ■ ■■■ auth.py ■ ■ ■■■ document_ingestion.py ■ ■ ■■■ rag_chat.py ■

■ services/ ■ ■ ■■■ __init__.py ■ ■ ■■■ auth.py ■ ■ ■■■ document_ingestion.py ■ ■ ■■■ rag_chat.py ■ ■■■ routers/ ■ ■ ■■■ __init__.py ■ ■ ■■■ auth.py ■ ■ ■■■ knowledge_bases.py ■ ■ ■■■ documents.py ■ ■ ■■■ chat.py ■ ■■■ rag/ ■ ■ ■■■ __init__.py ■ ■ ■■■ ingestion.py # Docling, chunking, embeddings ■ ■ ■■■ retrieval.py # pgvector search, reranking ■ ■ ■■■ generation.py # LLM prompt, response parsing ■ ■■■ utils/ ■ ■■■ security.py # JWT, password hashing ■ ■■■ logging.py ■■■ alembic/ Leverage Your S

■ ■■■ utils/ ■ ■■■ security.py # JWT, password hashing ■ ■■■ logging.py ■■■ alembic/ Leverage Your Strengths [3] [1] [2] 7. Sample Project Structure ■ ■■■ versions/ ■ ■■■ ... ■■■ tests/ ■ ■■■ __init__.py ■ ■■■ test_auth.py ■ ■■■ test_documents.py ■ ■■■ test_chat.py ■■■ .env ■■■ .gitignore ■■■ requirements.txt ■■■ Docker■le ■■■ README.md This project is an excellent choice for your ■nal year. It aligns with 2026 market demands, leverages your existing Python/PostgreSQL skills, and ■lls the gap in

igns with 2026 market demands, leverages your existing Python/PostgreSQL skills, and ■lls the gap in your AI/ML portfolio. By following this roadmap, you'll graduate with: Production-grade FastAPI backend experience Hands-on RAG/LLM implementation skills Understanding of vector databases, embeddings, and chunking strategies A portfolio piece that demonstrates both backend engineering and AI/ML competencies Start with Phase 0 this week. Build momentum with small, working increments. Document your

petencies Start with Phase 0 this week. Build momentum with small, working increments. Document your journey. You've got this. References: All market data, technology recommendations, and best practices cited from 2025- 2026 sources including vector database benchmarks, FastAPI/SQLAlchemy async patterns, RAG pipeline guides, document processing tools, and AI agent frameworks. ■ 1. https://clouddevs.com/job/toptal-python-backend-development-talent-with-rag-and-agentic-ai-experience/ 2. https://ww

s.com/job/toptal-python-backend-development-talent-with-rag-and-agentic-ai-experience/ 2. https://www.shine.com/jobs/senior-python-developer-backend-architecture-uttar-pradesh/sparix-global/19395926 3. https://www.mployee.me/jobs/view/python-react-full-stack-developer-agentic-ai-llm-exp-any-ust-location-immediate-joiner-in-bangalore-at-ust-4453534097/s1 4. https://www.linkedin.com/pulse/getting-started-fastapi-async-postgresql-step-by-step-omkar-kamat-uhbcf 5. https://atalupadhyay.wordpress.com

tarted-fastapi-async-postgresql-step-by-step-omkar-kamat-uhbcf 5. https://atalupadhyay.wordpress.com/2026/08/25/fastapi-200-interview-questions-answers-part-2-questions-101-200/ 6. https://www.querystack.tech/post/sqlalchemy-20-async-orm-patterns-pitfalls-and-production-ready-session-manage ment-5c4068 7. https://universopython.com/en/blog/python-sqlalchemy-async 8. https://chat2db.ai/resources/blog/sqlalchemy-postgresql-tutorial 9.

https://www.linkedin.com/posts/siddharth-mohril-361678250_datae

og/sqlalchemy-postgresql-tutorial 9. https://www.linkedin.com/posts/siddharth-mohril-361678250_dataengineeri
ng-postgresql-buildinpublic-activity-7491 173231140241408-PFC9 Closing Thoughts [1] [2] [3] [12] [18] [19]
[20] [21] [22] [31] [34] [42] [4] [5] [11] [17] [28] [40] [41] [7] [8] [10] [14] [15] [16] [23] [24] [27] [13] [26] [29] [30]
[36] [37] [6] 10. https://canpolatkaya.com/en/blog/sqlalchemy-2-async-and-alembic-migrations 11.
https://app.studyraid.com/en/read/119148/5501972/integrating-s

my-2-async-and-alembic-migrations 11.
https://app.studyraid.com/en/read/119148/5501972/integrating-sqlalchemy-async-orm-with-fastapi 12.
https://app.studyraid.com/en/read/119168/5503294/sqlalchemy-orm-basics-for-fastapi 13.
https://arxiv.org/abs/2608.12812 14. https://modelrefs.com/tools/best-vector-databases/ 15.
https://druce.ai/governance/wiki/comparisons/vector-database-landscape 16.
https://pratikdhanave.com/blog/posts/lc-05-retrieval-and-rag.html 17. https://flur.ee/blog/weaviate-alternati

ratikdhanave.com/blog/posts/lc-05-retrieval-and-rag.html 17. https://flur.ee/blog/weaviate-alternatives 18.
https://samaustinai.blogspot.com/2026/09/best-vector-databases-compared-2026.html 19.
https://mrscraper.com/blog/best-vector-database 20.
https://www.shipgarden.com/gallery/postgres-vector-database-nextjs-rag-2026 21.
https://www.promptquorum.com/power-local-llm/pinecone-vs-weaviate-vs-qdrant-vs-chroma-2026 22.
https://hussain-nazary.github.io/top-10-vector-databases-2026.html 23. https://

t-vs-chroma-2026 22. https://hussain-nazary.github.io/top-10-vector-databases-2026.html 23.
https://aiweekly.co/learning-ai/generative-ai/how-to-build-a-rag-pipeline 24.
https://blog.redlineso.net/posts/building-production-ready-rag-pipeline-llamaindex/ 25.
https://forage.ai/blog/langchain-vs-llamaindex/ 26.
https://machinelearningmastery.com/how-to-build-a-robust-rag-system-with-minimal-resources/ 27.
https://dev.to/priyeshdave6/langgraph-vs-crewai-vs-autogen-107-real-data-engineering-tasks-at-

/ 27.
https://dev.to/priyeshdave6/langgraph-vs-crewai-vs-autogen-107-real-data-engineering-tasks-at-scale-4h3e 28.
https://openprogram.io/docs/comparisons/ai-agent-frameworks.html 29. https://www.linkedin.com/posts/rajvard
han-patil-747a0422a_rag-llm-documentai-activity-7493586737718411264-qCR S 30.
https://www.context.dev/blog/how-to-scrape-websites-into-llm-ready-markdown-for-rag-pipelines 31.
https://tools.scult.in/skills/database/sqlalchemy-postgres 32. https://app.studyraid.com/en/read/11914

tps://tools.scult.in/skills/database/sqlalchemy-postgres 32.
https://app.studyraid.com/en/read/119148/5501984/issuing-and-refreshing-access-tokens 33.
https://app.studyraid.com/en/read/119148/5501982/implementing-oauth2-password-flow-with-jwt 34.
https://app.studyraid.com/en/read/119148/5501930/ensuring-statelessness-and-cacheability-in-rest 35.
https://mjmjmj.name/python/fastapi-jwt-oauth2-authentication/ 36.
https://khimananda.com/blog/rag-explained-retrieval-augmented-generation 37. https://p

tication/ 36. https://khimananda.com/blog/rag-explained-retrieval-augmented-generation 37.
https://pratikdhanave.com/blog/posts/nvidia-py-05-rag.html 38. https://www.llms.blog/posts/document-chunking
-strategies-in-production-rag-comparing-recursive-splitting-semanti
c-clustering-late-chunking-and-contextual-retrieval-architecture-boundary-preservation-and-retrieval- 39. https:/
/www.llms.blog/posts/chunking-strategies-in-production-rag-comparing-fixed-size-semantic-chunking-late-chu
nking-and-con

/chunking-strategies-in-production-rag-comparing-fixed-size-semantic-chunking-late-chu
nking-and-contextual-retrieval 40. https://preporato.com/tasks/rag-pipeline-build 41.
https://www.guideflow.com/blog/vector-search-soware 42.
https://www.scaler.com/topics/choosing-a-vector-database-for-rag/ 43.
https://note.com/bright_jacana710/n/n2d395a4f3aa8?hl=en 44.
https://github.com/anglosaberdev/llamaindex-rag-pipeline 45. https://metacpan.org/dist/PAGI-FastAPI-Security